

From BPMN to smart contracts on blockchains: Transforming BPMN to DE-HSM multi-modal model

Chris Liu
Faculty of Computer Science
Computer Science
Halifax, Canada
Chris.Liu@dal.ca

Peter Bodorik
Faculty of Computer Science
Computer Science
Halifax, Canada
Peter.Bodorik@dal.ca

Dawn Jutla
Sobey School of Business
Saint Mary's University
Halifax, Canada
Dawn.Jutal@gmail.com

Abstract— Scalability, privacy, and interoperability are some of the major issues receiving attention in research on blockchain technologies. We concentrate on the trade finance vertical for which we develop a new modeling approach with the objective of automatic transformation of an application, represented using Business Process Model and Notation (BPMN), into a smart contract deployed on a blockchain. Here, we describe how the BPMN model is transformed into a multi-modal model that combines DE-HSM modeling. We provide a high-level overview of the method and review how BPMN categories of elements are transformed into a multi-modal DE-HSM model. We also describe briefly how the DE-FMS model is automatically transformed into deployable smart contracts that interact to form a distributed application: The smart contract deployed on the main blockchain coordinates activities amongst the business partners and interoperates with smart contracts, also automatically prepared and deployed on a sidechain(s), with one smart contract per individual business partner. Privacy is obtained by performing activities, which are not germane to the collaboration with the other business partners but deal with the private activities of the individual business partner, in a smart contract deployed and executed on a side chain. We thus provide for interoperability of smart contracts and privacy as private activities of a business partner are performed in a smart contract on a private sidechain.

Keywords— *Blockchain, Business Processes Modeling Notation (BPMN), Discrete Event (DE) Modeling, Finite State Machine (FSM), Hierarchical State Machines (HSM), Smart Contract, Smart Contract Deployment, Off-chain computation, Privacy, Smart Contract Interoperability*

I. INTRODUCTION

Blockchain scalability is one of the prominent issues that researchers and practitioners have been addressing. One of the approaches in handling the scalability issues is moving computation and storage off-chain. Eberhardt and Tai [1,2] categorized methods, for reducing the blockchain size, based on what is moved to off-chain, into: (i) off-chain storage, (ii) off-chain computation, and (iii) a hybrid approach, in which both storage and computation are off-chain. All off-chain storage methods store an object off-chain but also store the object's hash code on the blockchain so that it can be used to ascertain its object's immutability any time that object is retrieved. In Plasma project [3], computation is performed off-chain while the blockchain is used for recording of certifications that off-chain computation is correct.

We explored how to use off-chain computation to mitigate the scalability issues in [4, 5] by investigating which parts of an application are suitable to be processed off-chain, under which conditions should processing off-chain be deployed, and how to facilitate deployment and processing off-chain.

We chose applications of the trade finance vertical for modeling purposes. We represent the control flow of the computation using Hierarchical State Machines (HSMs), which are Finite State Machines (FSMs) with states that themselves may represent FSMs. We devised an algorithm that finds *independent* subgraphs in a state graph that models an application (in [4], we refer to an *independent* subgraph as a *simple* subgraph). We consider independent subgraphs as suitable for processing off-chain due to their properties in that an independent (simple) subgraph has activities that are localized and hence suitable for processing off-chain. Local processing properties of independent subgraphs reduce interaction with the rest of the application and hence reduce the overhead costs. As off-chain processing incurs overhead, off-chain processing benefits in cost reduction should outweigh the overhead cost. We also demonstrated how to automatically transform an HSM model into methods of the (main) smart contract and deploy the smart contract, also automatically, on a main blockchain such that the main smart contract interacts with the independent subgraphs automatically deployed on a sidechain(s).

In [6], we explored how to model a trade finance application using multi-modal modeling that combines hierarchical modeling using HSMs with Discrete Events (DE) modeling [7] to form multi-modal DE-HSM modeling. We successfully model applications and then transform them into smart contracts wherein:

- Interactions activities between business participants is achieved using a smart contract running on the main blockchain.
- Activities pertaining only to an individual business partner are executed as a smart contract deployed and executed on a sidechain and hence the smart contract is private and not visible to the other participants. Furthermore, interoperability of smart contracts is also achieved as the smart contract on the main blockchain interoperates with the smart contracts deployed on a sidechain(s).

The consequence of the above is significant because our approach allows for (1) automatic transformation of a DE-HSM model into methods of smart contracts and (2) the main blockchain contract represents collaborative activities amongst the participants while independent activities of each participant are performed by a smart contract executed on a sidechain. That is, privacy is also achieved as the activities of a business participant can be executed privately on a sidechain.

However, DE-HSM modeling is not well-known and used and hence is a barrier to wide-spread adoption. Instead of using DE-HSM modeling to represent a trade finance

application, some higher-level of abstraction modeling approach should be used. And, in our opinion, there is not a more suitable modeling approach for such a purpose than Business Processes Modeling Notation (BPMN) modeling [8].

A. Our Goal and Approach, and Paper Objectives

1) Our Goal and Approach

Our goal is to allow for a trade finance application to be represented in BPMN and then, through a sequence of transformations, automatically convert the application's BPMN model into a smart contract(s) executing on a blockchain(s). The major step we take to our goal are:

- a) We start with BPMN modeling that represents the collaboration amongst the participants. We then transform the BPMN model into XML using the OMG specification on representing BPMN diagrams in XML specification [8].
- b) We transform the BPMN model (represented in XML [8]) into DE-HSM model, which is the subject of this paper. We then transform the model DE-HSM model into XML description.
- c) We use our algorithm to find independent subgraphs, in the state transition graph of the HSM model of collaboration, subgraphs that are suitable for execution off-chain — if the modeler chooses to do so [16].
- d) We automatically transform the DE-HSM model into methods of a smart contract executing on the main blockchain that has an interface to a smart contract for each business partners that is automatically prepared and deployed for execution on a sidechain. Each business partner is then free to provide details of the private activities that produce inputs to the main contract and consume outputs from the main contract [4, 5] execution of collaborative activities.

2) Paper Objectives

The objective of this paper is to describe the transformation of a BPMN model into an DE-HSM model, which is the first step in the above sequence of transformations.

B. Outline

In Section II, we provide background. Section III provides an overview of how a deployed DE-HSM model is executed while in Section IV we describe the process of transformation from BPMN to DE-HSM model. Section V provides a summary and concluding remarks.

II. BACKGROUND

In this section, we first review briefly the trade finance applications and their properties; we then describe DE-HSM multi-modal modeling that combines DE modeling for concurrency with HSM modeling; and then we conclude with an overview of BPMN elements.

A. Trade Finance Applications

Trade finance is a broad area that deals with the involvement of third-parties in trade for the purposes of payments, removing payment and other risks, and providing efficient processing of transactions by participants, such as

banks, customs, shippers, and insurance companies. Activities involved tend to be complex, long-term, and involving interaction and collaborations of many actors. We focus on collaborative activities that need to take place amongst many actors that communicate and interact only through messaging to exchange information, including information on links to files or data stores.

B. FSMs, Hierarchical State Machines, and Nested Multi-modal Modeling

FSM modeling is used frequently in the design and implementation of software. Variations of FSM modeling techniques were proposed, such as a guard along an FSM transition, a guard that specifies a Boolean condition on the state's variables that must be true for the transition to take place. In the late 80's, FSMs were extended to address the issues of reuse of patterns with the concept of hierarchy, leading to Hierarchical State machines (HSMs) that may contain states that are themselves other FSMs. An HSM can be defined using induction as follows (due to [9] and also described in [10] and [11]). In the base case, an FSM is a hierarchical machine. Suppose that M is a set of HSMs. If F is an FSM with a set of states, S , and there is a mapping function $f: S \rightarrow M$, then the triple (F, M, f) is an HSM. Each state, $s \in S$, that represents an HSM is replaced by its mapping $(f(s))$. HSMs recognize the same language as its corresponding flattened FSM. HSMs do not increase expressiveness of FSMs, only succinctness in representing them.

Because a single hierarchical state at one level can be considered to be in several states concurrently as represented by an FSM(s) in a lower level of the hierarchy, Harel [10] showed that FSMs can be combined hierarchically. FSMs may also be combined leading to concurrent FSMs. Girault et al. [9] describe how HSM modeling can be combined with concurrency semantics of a number of concurrency models, including communicating sequential processes [12] and discrete events [7]. They describe how an HSM model can represent a module of a system under a concurrency model that is applicable only if the system is in that state. This enables representation of a subsystem using a particular concurrency model that may be nested within a hierarchical state of a higher-level FSM - thus leading to a multi-modal modeling. In multi-modal modeling, different (hierarchical) states may be combined with different concurrency models that are best suitable for modelling activities of an FSM represented by an HSM for that particular state.

We target trade-finance applications in which organizations participate by exchange of messages. Furthermore, as previously mentioned, we exploit the concept of multi-modal modeling to allow the designer to model concurrent, but independent activities, by concurrent FSMs at the lower level of hierarchy, in order to avoid the state explosion that may arise in such situations

In [6] we showed how to create a multi-modal model that combines Discrete Event (DE) modeling with FSM modeling to model trade finance applications – such a combination of models is referred to as DE-HSM multi-modal modeling. We also showed that it is possible to automatically transform a DE-HSM model of such an application into methods of a smart contract deployed on a blockchain. In this paper, we address the issue of transforming a model, at a high-level of abstraction expressed in BPMN, into a DE-HSM multi-modal

model with the objective of transforming it into methods of smart contracts deployed on blockchains.

C. BPMN Overview

BPMN is an OMG standard that was developed to be understandable by all business users, from business analysts, through technical developers implementing the processes, to people managing those processes. It is viewed as a de-facto standard for describing business processes. For instance, Oracle Corporation uses the Oracle Business Process Analysis Suite to model business processes using BPMN and transform a BPMN diagram into a blueprints for executable Business Process Execution Language (BPEL) processes [13], wherein the blueprint represents the logic of the application in terms of concurrent processes and their interactions, while details of individual tasks are supplied by implementors. We are describing a similar approach, but instead of targeting BPEL, we target smart contracts executing on blockchains.

BPMN has four basic categories of elements: swim-lanes, which are referred to in BPMN documentation as swimlanes, flow elements, data, and artifacts. We briefly describe each category.

Swimlane: It represents a participant in a business process. There are two types of swimlanes: *Lanes* and *Pools*. A pool with lanes represents an organization while lanes within a pool represent individual participants within that organization. Only message flows can cross pools - no other BPMN elements can do so.

Flow Elements: They are used to form a process flow and they include: *Task* and *Sub-Process*, which are referred to as *activities*, and *Events* and *Gateways*. These elements are connected either by *sequence flows* or by *message flows*. A task represents an atomic activity that is not broken down any further. A sub-process is used to represent complex work that is further broken down into activities represented by a Business Process Diagram (BPD) using BPMN. Any level of nesting may be obtained. The restriction is that BPMN elements within a sub-process may be connected only to other elements within the same sub-process and are thus localized to the sub-process that contains them. Subprocesses may be executed in a controlled loop and/or may also represent multiple instances of the subprocess being executed in parallel.

Events: There are three types of events: *Start*, *Intermediate* and *End*. Start and End events are special in that they denote the start and an end of an activity or a business process. Intermediate events include many different types with examples including message timer, timer, conditional, escalation, and multiple parallel. Each type has a special symbol with an icon having special marking to indicate properties, such as throwing or catching, interrupting and non-interrupting, boundary, and non-interrupting. Throwing events are connected to catching events with sequence flow to show the flow of control.

Gateways: They are used to represent the flow of a business process. A gateway may have one or more incoming flows and one or more outgoing flows to represent merging and splitting of flows, respectively. Merging of flows may be pass-through, i.e., without synchronization, or with synchronization to ensure that all active incoming flows are present before the outgoing flows may proceed. Outgoing

flows may be inclusive, i.e., parallel, or exclusive in which case only one of the flows is allowed. Furthermore, control of which outgoing flows may proceed may be controlled by data or gateways. As any data-based split or merge can be represented by a gateway, we first transform the data-based merges and splits in a straight-forward way to gateways.

In addition to the sequence and message flows, there are also *Associations* and *Data Associations*. A Data Association is used to show the flow of information in the diagram. Association, on the other hand, may be used to provide any additional details about a flow element.

Data Objects: They are used to show data that is input into or produced by a flow element, in which case Data-input and Data-output icons are used. There is also a *Data store* element.

Choreography: In the version 2.0.2, OMG introduced choreography in BPMN to represent the way business participants coordinate their interactions. The focus is on the sequence of information exchanged via messages as they interact. Each message exchange represents either a single request-reply type interaction or a conversation that involves an exchange of several messages to achieve the desired goal.

III. BPMN TO DE-HSM: TRANSFORMATION OVERVIEW

We first provide a high-level overview that describes how the BPMN model is transformed into the DE-HSM model using a simple example. In the subsequent section we describe how individual BPMN elements are transformed into the DE-HSM model in more details. Due to the space limitation, we provide general description only while omitting many details.

A. Overview of the Deployed App Execution

Although we are describing the transformation of the BPMN into the DE-HSM model, we need to understand and hence review how the result of the deployed transformation of the DE-HSM model is executed by the blockchain infrastructure.

In a deployed and executing application, participants can perform their own actions concurrently with the those of the other participants. Thus, each participant has its own FSM. External signals, e.g., calls to a smart contract method, are translated into events that are stored in a queue, called Q, of events that are timestamped using a global logical clock with the oldest event at the head of the queue. Fig. 1 shows the queue Q and FSMs, one for each participant. In addition, there is also an FSM that corresponds to the system. Its role is to represent activities that deal with the collaboration amongst the participants, while the rest represent activities of individual business partners.

Any external event, such as a participant sending or receiving a message, is transformed into an event that is queued into Q for processing. Execution of the application proceeds by removing, from the priority queue Q, one event at a time and processing it as outlined below.

The dequeued event is analyzed and is handed over for processing to one of the participants FSMs determined based on the event's properties. Then another event is dequeued and the process repeats. A participant who receives a dequeued event for processing, first analyzes it and transforms it inputs to its FSM. Depending on the current state and input, the FSM

makes a transition, while producing output that is returned to the DE component of the model. The DE component translates the FSM's output, and responds to the external event as is appropriate, such that the response may also cause an event to be queued into Q for further processing. Thus, the BPMN transformation needs to describe how the external signals are transformed into inputs into one of the FSMs, where each FSM represents the control and data flow for its corresponding participant. Furthermore, the DE and FSM modeling combined must represent the control and data flow as represented by the BPMN diagram and the transformation needs to prepare both.

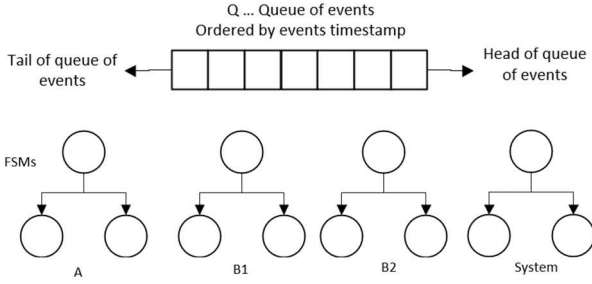


Fig. 1. Queue Q of DE events and FSMs for participant

B. Overview: BPMN Transformation into DE-HSM model

We shall use a simple BPMN diagram, shown in Fig. 2, as an example. The diagram shows a client A, represented by a pool, and a seller B, represented by a pool with two lanes, one for the agent B₁ and the other for accounting department, B₂. The client first prepares an order (task *PrepReq*) that is messaged to B₁. Then B₁ asks accounting to prepare an invoice (task *ReqInvoice*) and then packages the product. Once both the package is ready and the invoice has been received from accounting, both are messaged to the client. Note that a message is represented by two events, one for sending and one for receiving the message.

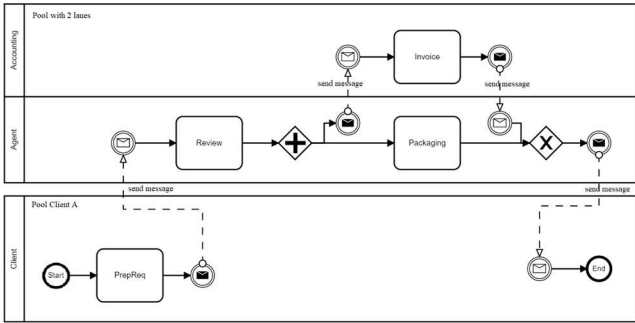


Fig. 2. BPMN diagram of client A interacting with the product provider B

In general, we may have a DE-FMS model that, itself, may have inner, also referred to as a child, DE-HSM models and, furthermore, they can be nested recursively, each one with its own clock, queue of events, and FSM model.

As our child DE-Models are such that they do not have circular dependencies, we can assume that all child FSMs have a zero-delay execution for their DE model and thus we need only to use one global logical clock for timestamping and only one queue of events, Q, that is ordered in the timestamp priority [14]. Consequently, we have one priority queue of

events, with the oldest event being at the head of the queue, that is shared by all DE models (as already shown Fig. 1).

Once the FSMs and the queue Q are set up, we start analyzing each of the BPMN elements, starting with the *Start* event, in order to expand the participant FSMs according to the BPMN diagram elements. The *Start* event has a flow connection to the client that performs the task *PrepReq*. As a result, a new state, *PrepReq*, is added to the FSM_A, that is the client's FSM, as is shown in Fig 3(a). It should be noted that we are preparing a blueprint for the application, blueprint that represents the flow of control and data representing interactions amongst the participants, between lanes and across pools. For each task, which is an atomic unit in BPMN, we prepare a call to the method with input and output parameters as represented by the BPMN diagram using either data input and output elements, or via associations. As BPMN does not require such information, we ask the modeler for such information, if it is not represented in the BPMN diagram already, and we amend the diagram accordingly. Only then we perform the transformation into the DE-HSM model.

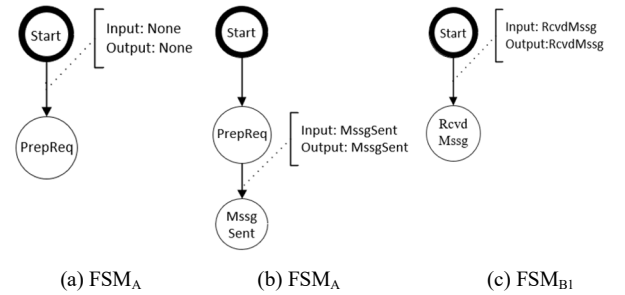


Fig. 3. Transforming flow elements and tasks

As there are no further outgoing connections for the *Start* event, we now follow elements outflowing from the *PrepReq* element that represents sending to B₁ a message that contains the request for a product. Sending a message is recognized by two events, one when the message is actually sent, and one when it is actually received, both generated due to external signals. When executing, as these externally triggered events are caught, the corresponding events are queued into Q for processing. Transformation thus augments the FSM_{B1} with the state *MsgSent* corresponding to the sent-message event as shown in Fig. 3(b). As there are no further connections emanating from the *SendMsg* BPMN element, we follow its outflowing connection to the *Review* task and we augment the FSM_{B1} with a state *RcvdMsg*, as shown in Fig. 3(c).

We then follow the outgoing flow connections from the *Review* element that leads to a gateway that performs a parallel split. If the gateway had guards (Boolean expressions) on outflowing sequence flows, they would be analyzed and would result in a guard condition associated with a transition for each of the outgoing flows of the gateway. As the gateway is parallel, we create two additional concurrent FSMs, FSM_{B11} and FSM_{B12}, each one corresponding to a DE-HSM model nested within FSM_{B1} to represent the two paths of concurrent activities emanating out of the gateway. As DE models for each of FSM_{B11} and FSM_{B12} have zero delay, we need not create a new queue of events for each of the two models, but rather utilize the main queue of events, Q. The two new, concurrent FSMs are shown in Fig. 4. Furthermore, FSM_{B1} is

modified to guide an input signal to one of the concurrent FSMs, FSM_{B11} or FSM_{B12} , as is shown in Fig. 5(a). Finally, an event generation (event generated at execution time) is prepared for each of the two paths, one to *PrepProduct* and one to *PrepInvoice*, that will generate corresponding events that are queued into Q during execution. The software then follows the outgoing flow connections from the gateway, one at a time. For the flow connection to the activity *PrepProduct*, the software amends the FSM_{B11} with the state *PrepProduct* to represent the activity of preparing the invoice. The input is the request for product and the output is the prepared product. Furthermore, when executed, an event *PrepProduct* is queued into Q. The connection flow to *PrepInvoice* is transformed similarly, resulting in FSMs FSM_{B11} and FSM_{B12} being amended as shown in Fig. 5(c).

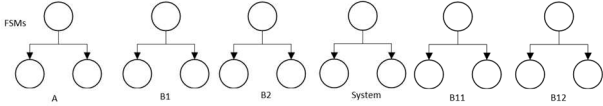


Fig. 4. Two FSMs added, FSM_{B11} and FSM_{B12} , are amended to represent concurrent activities

IV. TRANSFORMING BPMN DIAGRAM ELEMENTS

The transformation proceeds in three phases, *initial*, *middle*, and *completion*. In the initial phase, the priority queue and skeleton FSMs are prepared and every sub-process in the BPMN diagram is expanded into its basic BPMN activities. Finally, any BPMN data-based merges or splits are replaced in the BPMN diagram with equivalent gateway constructs. A breadth-first search is performed from the *Start* event along the outgoing connections to analyze each of the diagram's BPMN elements and transform the BPMN diagram into DE-HSM model to build actor FSM(s). The process repeats for each of the BPMN icon along the outgoing flow connections, in the breadth-first-search pattern. How the FSMs are built and

how DE events are scheduled depends on the BPMN element type. We briefly describe each of the phases.

A. Transformation Phases

1) Initial Phase: Queue of DEs and Participant FSMs

We start with pre-processing of the BPMN diagram. First, the BPMN's choreography, which is used to represent message conversations via message exchanges amongst actors, is transformed into a BPMN diagram in which details of conversations are provided by the modeler and used to transform the conversation into activities described using the core BPMN elements. Next, each sub-process is expanded into its constituent BPMN elements as per the sub-process descriptions. Finally, we transform any data-based merging and splitting of flows into equivalent gateway constructs.

Once the preparation of the BPMN diagram is completed, the priority queue of discrete events, Q, is constructed. The BPMN diagram is then analyzed and for each pool with no lanes and for each lane in a pool, an FSM is constructed to represent the activities of individual participants as they may proceed concurrently with activities of the other participants – as was already described in the previous sub-section and illustrated in Fig. 1.

2) Middle Phase: Transform BPMN Elements

Although BPMN has many icons/elements, they can be grouped into categories, or sub-categories, such that the same transformation process may be applied with some variations that result in guards on FSM transitions to represent branching in a BPMN diagram using gateways (and recall that we transform data-based merges and splits into equivalent constructs using gateways). In general, any outgoing flow that results in a split of flow is transformed by adding one FSM for each of the concurrent outgoing flows. Once concurrent activities are merged, FSMs that represent such activities become inactive.

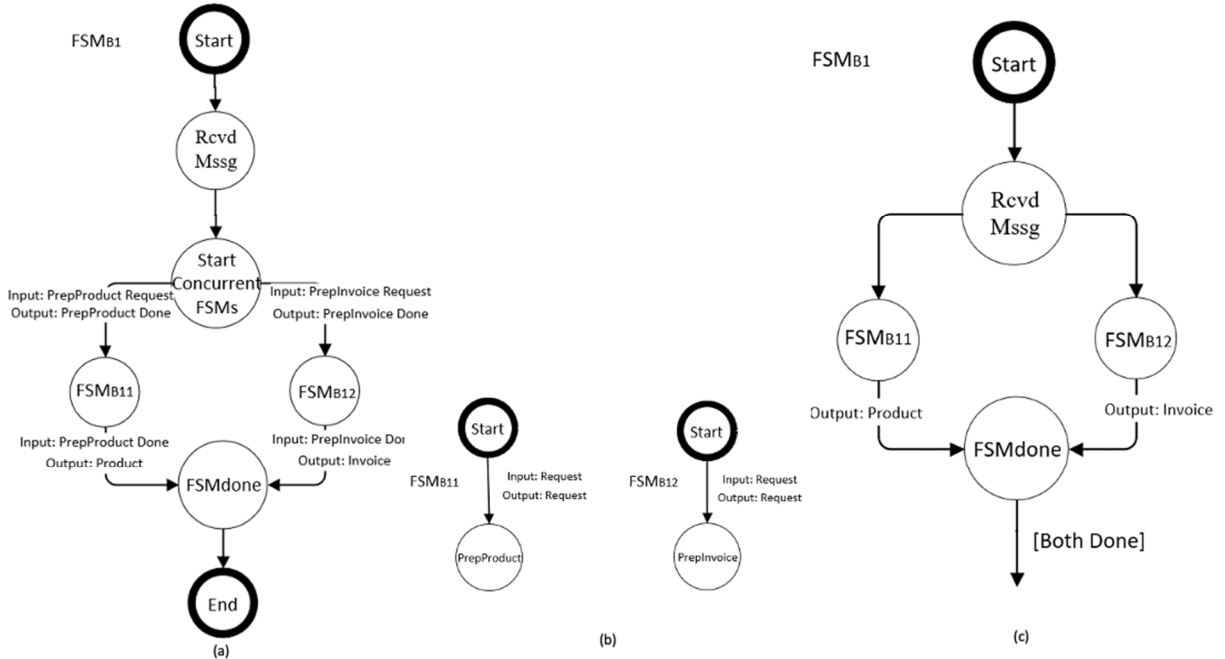


Fig. 5 Transforming a BPMN split gateway leading to concurrent FSMs

For any outgoing *Sequence flow* connection that does not result in concurrent FSMs, if the element to which it connects is an activity, the FSM for the participant is amended with a new state, representing that activity, and a transition to it. During execution, when an FSM transitions into a state that represents a task, it results in an invocation of a blue-print method with input and output parameters that are formed using the inputs and outputs of the transition. We prepare a skeleton, called a blueprint, for such a method that represents a task, but its elaboration is left to the developer – hence, we develop a blue-print for a distributed application with the flow of control and the flow of data as per the execution of the DE-HSM model that, in itself was prepared by transformation from a BPMN model. The details that represent localized activities represented by tasks is left to the software developer.

3) Final Phase: Transformation into XML

In the last phase, we transform the DE-HSM model into XML representation that is then used for further transformations that result in the methods of smart contracts that are deployed on the mainchain and sidechains.

B. Transforming BPMN Elements Categories

We briefly outline the transformation for each BPMN category of elements.

Pools and Swimlanes: As discussed above, for each pool without lanes and each lane of a pool we prepare an FSM that is amended by processing of elements as described below.

Connecting Objects: *Sequence flow* and *Message flow* are used to describe the ordering of the various activities and their transformations that depend on the type of the BPMN element to which flow connects as per below description.

Tasks and Sub-processes: As stated before, sub-processes are expanded in the initial phase into the BPMN basic elements as per the sub-process description. Each task results in a new FSM state for the participant that represents the swimlane in which the task activity is located. A task is transformed into a blue-print method as discussed above.

Gateways: A gateway may have one or more incoming and outgoing flows to represent the merging and splitting of flows, respectively.

Merging of flows: Merging may be synchronized, meaning that all incoming flows must complete (arrive) before any outgoing flow proceeds, or may be pass-through, i.e., without synchronization. For a pass-through gate, outgoing flows are tested to see if they may proceed (in case of guards on the outgoing flows). If the gate inflows are synchronized, then there is an additional state amended to the FSM of the participant whose swimlane contains the gateway. The state represents the condition that all incoming flows to the gate have arrived and hence there is a corresponding guard (Boolean expression representing that all flows have arrived) that is attached to the newly added state.

Splitting of flow: Outgoing flows may be inclusive, i.e., parallel, or exclusive in which only one of the flows is allowed. If the outgoing flows are inclusive, then a new concurrent FSM is added for each participant that has a lane containing a BPMN element to which the outgoing flow is connected. Recall that outgoing flow may not cross pools unless (i) the connection is outgoing from a message-sent

event to a receive-message event in another pool or (ii) it is a connection from a thrown event to receive event in another pool. The FSM for the participant whose lane contains the gateway is also amended correspondingly to channel incoming signals to one of the concurrent FSMs.

If, however, the outgoing flows are exclusive, the new state is amended to the FSM of the participant containing the gateway for each of the outgoing flows, such that each has a guard specifying the exclusive condition under which computation may proceed along that path.

Data Object: Access to a Data Object is represented in a similar way as a task is. We prepare a skeleton/stub for the getter and setter of the data in the object, but we do not provide details of its implementation, which is left for the developer.

V. SUMMARY AND CONCLUDING REMARKS

In this paper we described our approach of transforming trade finance applications, represented using BPMN modeling, into a DE-HSM model. We first provide motivation for our work by describing how this transformation is used to achieve the goal of transforming DE-HSM model into a smart-contract methods, wherein collaborative activities are executed by the smart-contract methods executed on the main blockchain, while business partner activities are executed by a smart contract executed on a sidechain. After the BPMN model of a distributed application is transformed into the DE-HSM model, subject of this paper, the DE-HSM model is analyzed and automatically transformed into the smart contracts that are deployed on a blockchain(s). As our transformation provides for interaction amongst the smart contracts and their deployment, we thus provide for (i) interoperability of smart contracts in a distributed application and (ii) for privacy as activities local to an individual business partner are performed on a sidechain.

Due to the (page) space limits we provided only a general description while omitting many details. However, we are not only writing about the work but have implemented a Proof of Concept (PoC) - interested readers may view a demonstration of our example application, which is shown in Fig. 2, by using the following link <https://quinpool.com:19000/>. The demonstration shows the sequence of transformation, while also showing intermediate models and their XML representation. Finally, the smart contracts are also deployed automatically, and their execution is shown.

Of course, our current implementation is only a proof of concept, and many issues still need to be addressed. For instance, the use of a global logical clock to timestamp events that are stored in the timestamp-priority ordered queue (Q) leads to issues that need to be addressed. As execution consists of removal of events from Q and processing them, issues of efficiency and scalability need to be addressed. We note that the properties of finance trade applications are such that the speed of processing is not a priority as the transactions in this vertical are relatively long-term. However, our current investigation into this issue leads us to believe that good concurrency is attainable. The issue of scalability may arise due to the increase in the complexity of applications and due to the increase in the number of applications that need to be supported – also an issue under our investigation.

However, the most pressing issue is that of transactions. The BPMN standard defines a transaction and any blockchain also has a definition of a transaction(s), but the two definitions are not necessarily aligned. A blockchain transaction defines the basic atomic unit that is stored on the blockchain. The first question that arises is how to map a transaction, as defined and used in the context of the distributed application, into a transaction or transactions of the blockchain used as the infrastructure for the smart contracts deployment while also enforcing properties of the application definition of a transaction as specified by the BPMN. In BPMN, a transaction is a type of a sub-process that inherits from the sub-process objects but also has specialized property of “all or nothing”: Either sub-process activities complete successfully, or a compensating transaction must be used in case of problems and the transaction is cancelled. It should be noted that that as sub-processes may be nested in BPMN, their transactions may be nested also. Complexities further arise due to the BPMS *Data type* elements that are used to represent inputs and outputs to other BPMN components, or may represent data objects, or data stores.

REFERENCES

- [1] J. Eberhardt & S. Tai (2017). On or Off the Blockchain? In: De Paoli F., Schulte S., Broch Johnsen E. (eds) Service-Oriented and Cloud Computing. ESOC 2017. Lecture Notes in CS, 3-15, vol 10465, Springer.
- [2] J. Eberhardt, H. Jonathan (2018) Off-chaining Models and Approaches to Off-chain Computations. In Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers (SERIAL'18). 7-12.
- [3] J. Poon & V. Buterin (2017). Plasma: Scalable Autonomous Smart Contracts. <https://plasma.io/plasma.pdf> on Nov. 12, 2020. 47 pages.
- [4] P. Bodorik, C. Liu, D. Jutla, 2021. Using FSMs to Find Patterns for Off-Chain Computing: Finding Patterns for Off-Chain Computing with FSMs. In 2021 The 3rd International Conference on Blockchain Technology (ICBCT '21), March 26–28, 2021, Shanghai, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3460537.3460565>
- [5] C. Liu, P. Bodorik, D. Jutla, 2021. A Tool for Moving Blockchain Computations Off-Chain. In Proceedings of 2021 3rd ACM International Symposium on Blockchain and Secure Critical Infrastructure (BSCI'21), June 3–7, 2021, Virtual Event, Hong Kong. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3457337.3457848>
- [6] C. Liu, P. Bodorik, D. Jutla. 2021. Blockchain Privacy, Scalability, and Separation of Concerns Using Multi-modal Modeling. Accepted by Fourth International Conference on Blockchain Technology and Applications (ICBTA'21), December 17th-19th, 2021. ACM, New York, NY, USA, 15 pages. <https://web.cs.dal.ca/~gang/ICBTA/>
- [7] C. Cassandras, Discrete Event Systems, Modeling and Performance Analysis. Irwin: Homewood IL, 1993.
- [8] BPMN. Business Process Model And Notation. (2021). Retrieved from <https://www.omg.org/spec/BPMN/2.0/About-BPMN/> on July 09, 2021.
- [9] A. Girault and B. Lee and E.A. Lee (1999). Hierarchical Finite State Machines with Multiple Concurrency Models. IEEE Transactions on Computer-Aided Design, June 6, 1999. 742--760
- [10] D. Harel (1987). Statecharts: a visual formalism for complex systems. Science of Computer Programming, Volume 8, Issue 3, 1987, Pages 231-274, ISSN 0167-6423. DOI: [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9).
- [11] M. Yannakakis (2000) Hierarchical State Machines. In: van Leeuwen J., Watanabe O., Hagiya M., Mosses P.D., Ito T. (eds) Theoretical Computer Science: Exploring New Frontiers of Theoretical Informatics. TCS 2000. Lecture Notes in CS, vol 1872. Springer. 315-330.
- [12] C. A. R. Hoare, “Communicating sequential processes,” Commun. ACM, vol. 21, no. 8, Aug. 1978.
- [13] L. Dikmans (2008). Transforming BPMN into BPEL: Why and How. Retrieved from <https://www.oracle.com/technical-resources/articles/dikmans-bpm.html> on August 19, 2021.
- [14] Wikipedia. 2021. Lamport timestamp. WikipediA: the Free Encyclopedia. Retrieved from https://en.wikipedia.org/wiki/Lamport_timestamp.